PROJECT TEAM MATCHER

An Intelligent Team Formation Platform

SOFTWARE ENGINEERING PROJECT PROPOSAL

Course	UCS503P
Submitted to	Kritika Goel
Department	Computer Science and Engineering
Institute	Thapar Institute of Engineering and Technology, Patiala

Project Team

Mohit Garg	1024031006
Paras Sharma	1024030609
Manan Dhingra	1024030222
Parv Panwar	1024030226

Project Type	Software Engineering Project
Development Period	August – October 2026
Platform	Web Application
Department	Computer Science and Engineering

August 2026

1. Introduction

Project Team Matcher is a web-based team formation platform designed exclusively for students of Thapar Institute of Engineering and Technology. The system helps students find and form suitable teams for hackathons, college projects, competitions, startups and other collaborative activities.

In a college environment, students often have the required skills for a project but may not know other students who have complementary skills or are currently looking for a team. Team formation is frequently handled through informal groups and messages, which can make it difficult to discover available teams and identify the skills they require.

Project Team Matcher provides a centralized platform where verified Thapar students can maintain their profiles, create teams, discover available teams and request to join them. Team leaders can specify the number of students required and the skills needed for a project. Students can use filters such as hackathon and skills to find relevant opportunities.

The platform is intentionally kept simple while maintaining a sound software architecture, reliable database operations, access control, efficient searching and appropriate handling of concurrent requests.

2. Problem Statement

Students participating in hackathons, projects and competitions often face difficulty in finding teammates with the required skills. Existing communication channels may not provide a structured way to discover incomplete teams or compare opportunities based on skills and hackathons.

A team leader may also need to repeatedly ask in different student groups for members, while students who are looking for teams may not know which teams currently have openings.

The proposed system addresses these problems by:

- providing a Thapar-only platform for team formation;
- verifying students through their Thapar Student ID;
- maintaining a structured student profile with resume and professional links;
- allowing team leaders to specify required team size and skills;
- displaying incomplete teams to eligible students across campus;
- allowing students to filter teams by hackathon and skills;
- allowing students to request to join teams;

- making completed teams private; and
- maintaining reliable and consistent team membership data.

3. Objectives

The main objectives of Project Team Matcher are:

- Allow only Thapar students to register and access the platform.
- Authenticate students using their Student ID.
- Allow students to create profiles containing resume, phone number, GitHub and LinkedIn details.
- Allow students to create or join teams.
- Allow team leaders to define required team size and required skills.
- Display incomplete teams as available opportunities across the campus.
- Provide filters for hackathons and skills.
- Allow team leaders to review and manage join requests.
- Automatically mark a team as private when its required size is reached.
- Provide a dashboard showing current projects and available teams.
- Maintain secure, reliable and scalable backend and database operations.

4. System Workflow

The system workflow consists of four main stages.

4.1 4.1 Student Registration and Profile

A student registers using a valid Thapar Student ID. After authentication, the student can maintain a profile containing:

- Resume
- Phone number
- GitHub profile
- LinkedIn profile

- Skills and relevant interests

The profile provides the information needed by team leaders when evaluating potential members.

4.2 4.2 Team Creation

A student can create a team and act as its leader. During team creation, the leader specifies:

- Project or hackathon name
- Basic project description
- Number of students required
- Skills required

The newly created team remains available to eligible students while it has open positions.

4.3 4.3 Team Discovery and Joining

Students can browse available teams from the dashboard. Teams can be filtered using criteria such as hackathon and required skills.

The basic workflow is:

Login → Dashboard → Filter/Search Teams → View Team → Request to Join

The team leader reviews requests and accepts suitable students. The backend verifies the current team capacity before adding a member.

4.4 4.4 Team Completion and Privacy

When the number of accepted members reaches the required team size, the team is marked as completed. A completed team is no longer available for new students to join and becomes private.

Members can continue to see the project through the "Current Working Projects" section of their dashboard.

5. Team Capacity and Conflict Handling

Team capacity is a central part of the system. Before accepting a student, the backend checks:

1. whether the team exists and is still open;
2. whether the student is already a member;
3. whether the required team size has already been reached; and
4. whether the join request can be accepted according to the team status.

These checks must be performed on the server rather than relying only on the frontend. Database constraints and transaction handling can also be used to reduce the possibility of two simultaneous requests exceeding the team capacity.

For example, if a team requires five students and currently has four members, only one additional student should be successfully accepted even if multiple requests are processed at nearly the same time.

6. System Architecture

The system follows a client-server architecture.

Component	Responsibility
HTML5, CSS3, Vanilla JavaScript	Student dashboard, profiles, team creation, team discovery and joining interfaces
Node.js / Express.js.js	REST APIs, authentication, authorization and business logic
MySQL	Student, profile, skill, team, project and join-request data
Authentication	Student ID based authentication and secure sessions/JWT
Deployment	Containerized/cloud-hosted web application

The web application communicates with the backend through REST APIs. The backend handles authentication, authorization, team management, filtering, capacity validation and database operations.

The architecture is designed so that frontend requests do not directly access the database. This separation improves maintainability and allows the backend to validate important operations centrally.

7. Technology Stack

Area	Technology
Frontend	HTML5, CSS3, Vanilla JavaScript
Backend	Node.js, Express.js
Database	MySQL
API	REST API
Authentication	Secure sessions / JWT
Testing	Jest, Postman
Version Control	Git, GitHub
Deployment	Docker / cloud host

The frontend will be implemented using HTML5, CSS3 and Vanilla JavaScript without React, Next.js, TypeScript, Tailwind CSS, Bootstrap or any other external frontend framework. The frontend will be structured cleanly so that mock JavaScript data can later be replaced with API calls to the Node.js and Express.js backend.

8. Main Modules

The proposed system consists of the following modules:

- **Authentication Module:** Student ID based registration, login and access control.
- **Student Profile Module:** Resume, phone, GitHub, LinkedIn and skill information.
- **Team Module:** Team creation, team details, required size and required skills.
- **Discovery Module:** Display of available teams and current working projects.
- **Filter Module:** Filtering teams using hackathons and skills.
- **Join Request Module:** Requests to join teams and leader-side acceptance or rejection.
- **Team Status Module:** Open/incomplete teams and automatic transition to private completed teams.
- **Dashboard Module:** Central interface for creating, joining and tracking teams.

9. Database Design

The main entities are:

- User / Student
- StudentProfile

- Skill
- StudentSkill
- Team
- TeamSkill
- Project / Hackathon
- TeamMember
- JoinRequest

The basic relationships are:

Student $\leftrightarrow$ **TeamMember** $\leftrightarrow$ **Team**

Student $\leftrightarrow$ **StudentSkill** $\leftrightarrow$ **Skill**

Team $\leftrightarrow$ **TeamSkill** $\leftrightarrow$ **Skill**

A team is associated with a project or hackathon and contains multiple student members. Join requests connect students with teams before membership is confirmed. The database should enforce unique student IDs and prevent duplicate team memberships.

10. Functional Requirements

1. Users should be able to register and log in using a valid Thapar Student ID.
2. The system should restrict access to eligible Thapar students.
3. Students should be able to create and update their profiles.
4. Students should be able to upload a resume.
5. Students should be able to provide phone, GitHub and LinkedIn details.
6. Students should be able to create teams.
7. Team leaders should be able to specify team size and required skills.
8. The system should display incomplete teams to eligible students.
9. Students should be able to search and filter available teams.
10. Students should be able to filter teams by hackathon and skills.
11. Students should be able to send requests to join teams.

12. Team leaders should be able to accept or reject join requests.
13. The system should prevent team membership from exceeding the required capacity.
14. Completed teams should become private.
15. Students should be able to view their current working projects from the dashboard.

11. Non-Functional Requirements

- **Usability:** Team discovery and joining should require minimal steps.
- **Performance:** Search, filtering and dashboard requests should be processed efficiently.
- **Reliability:** Team membership and project data should remain consistent during concurrent requests.
- **Security:** Students should only access information and operations permitted to them.
- **Scalability:** The system should support a large number of Thapar students and teams without requiring major architectural changes.
- **Maintainability:** Frontend, backend and database components should remain modular.
- **Availability:** The application should be designed to remain usable during periods of high student activity.

12. Security

The backend will validate important operations rather than relying only on the frontend.

The system will use:

- Student ID based authentication;
- secure session or token handling;
- role and ownership based authorization;
- protected REST APIs;
- server-side input validation;
- file validation for uploaded resumes;
- database constraints for student and team consistency; and
- access checks for private completed teams.

For example, a student must not be able to modify another student's profile or change a team's membership through a direct API request without the required authorization.

13. Testing

Testing will be carried out throughout development.

- **Unit Testing:** Team capacity, filtering and validation functions.
- **API Testing:** Authentication, profile, team and join-request endpoints.
- **Integration Testing:** Frontend, backend and database interactions.
- **System Testing:** Complete student registration-to-team-completion workflow.
- **Security Testing:** Unauthorized access, invalid Student IDs and private-team access.
- **Load Testing:** Multiple concurrent users performing searches, team views and join requests.
- **User Testing:** Common student workflows such as creating and joining a team.

Important test cases include simultaneous join requests for the final team position, joining an already completed team, duplicate membership requests, invalid Student IDs, unauthorized team modifications and filtering with large numbers of teams.

14. Feasibility

Project Team Matcher is feasible as an academic software engineering project because it uses established web technologies and does not require specialized hardware.

The core functionality consists of authentication, profile management, database management, team creation, filtering, join-request processing and team-capacity handling. The main technical challenge is maintaining consistent team membership when multiple students attempt to join the same team simultaneously. This can be addressed through server-side validation, database constraints and transaction handling.

The first version can use a controlled Thapar Student ID validation mechanism rather than depending on complex institutional integrations. This keeps the project practical while leaving room for future integration with official college authentication systems.

15. Development Plan

The project will be developed in the following stages:

1. Requirements analysis and system design.

2. Database and backend setup.
3. Student ID authentication and access control.
4. Student profile and resume management.
5. Team creation and team-detail management.
6. Available-team discovery and filtering.
7. Join requests and leader-side team management.
8. Team capacity validation and private-team transition.
9. Dashboard integration.
10. Integration, security, load and system testing.
11. Final deployment and documentation.

The planned development period is August–October 2026.

16. Expected Result

The expected result is a working web-based Project Team Matcher prototype through which Thapar students can create profiles, create teams, discover available teams, filter opportunities by hackathons and skills, and request to join teams.

Team leaders should be able to define the number of students required and the skills needed for their projects. Once a team reaches its required size, it should become private and appear as a current working project for its members.

The system should demonstrate reliable authentication, modular architecture, efficient database operations, appropriate security and stable behaviour under concurrent usage.

17. Conclusion

Project Team Matcher provides a centralized solution for forming student teams within Thapar Institute of Engineering and Technology. The system addresses the practical difficulty of finding suitable teammates for hackathons, projects, competitions and other collaborative activities.

By combining verified student profiles, team creation, skill and hackathon based discovery, join requests, capacity management and automatic privacy after team completion, the platform provides a structured workflow for student team formation.

The project also provides an appropriate scope for demonstrating software engineering concepts including requirements analysis, system architecture, database design, authentication, authorization, modular development, validation, concurrency handling, testing, scalability and deployment.